

Teoria-DS.pdf



SpeedyALIF



Diseño del Software



3º Grado en Ingeniería Informática del Software



**Escuela de Ingeniería Informática
Universidad de Oviedo**

DISEÑO DEL SOFTWARE

Grado Ingeniería Informática Software

Curso 18/19

Contenido:

DISEÑO DEL SOFTWARE – TEORÍA ORIENTACIÓN A OBJETOS	3
1. Elementos básicos Orientación a objetos:.....	3
2. Interfaces y clases abstractas:	5
3. Principios de diseño Orientado a objetos:.....	7
DISEÑO DEL SOFTWARE – TEORÍA PATRONES	12
5A. Strategy:	12
5B. Abstract Factory:	14
6. Decorator:.....	17
7A. Adapter:.....	19
7B. Composite:.....	21
8A. Command:	23
8B. State:	25
8C. Template:.....	27
9A. Observer:	29
DISEÑO DEL SOFTWARE – PRÁCTICAS	30
Práctica 1: Videoclub:.....	30
Práctica 3: Editor gráfico:	32
Práctica 4: Formularios:.....	35
Práctica 5: Rehacer/Deshacer:.....	37
Práctica 6: Mapa:	39
Práctica 7: Sistema de ficheros:	40
Práctica 8: Sistema de encuestas:.....	42
Práctica 10: Ball game:	44
Práctica Simulacro examen:	47

DISEÑO DEL SOFTWARE – TEORÍA ORIENTACIÓN A OBJETOS

1. Elementos básicos Orientación a objetos:

- Diseño: pone énfasis en una solución conceptual que satisfaga los requisitos de un problema.
- Diseño orientado a objetos: presta especial atención a la definición de objetos software y cómo colaboran para satisfacer los requisitos.
- Resultado del diseño:
 - Sistema de objetos que satisfacen requisitos.
 - Descripción del comportamiento público de esos objetos.
 - Cómo se comunican esos objetos entre sí.
- Orientación a objetos: el diseño orientado a objetos construye las aplicaciones mediante una estructura de objetos que colaboran entre sí. Cada objeto realiza unas operaciones que lo caracterizan.
- Elementos de un objeto:
 - Está compuesto de datos y operaciones que operan sobre dichos datos (métodos).
 - Lleva a cabo una operación cuando recibe una petición o mensaje de un cliente:
 - Mensaje es la única forma de decirle a un objeto que ejecute una operación.
 - Las operaciones son el único modo de cambiar el estado interno del objeto.
 - Por ello, se dice que el estado interno del objeto está encapsulado: no se puede acceder a él directamente y su representación es invisible desde el exterior del objeto.
- Clases: especificación de la implementación de los objetos:
 - Un objeto se define por su clase: es la implementación del objeto.
 - Definición de clases:
 - Determinar el nombre a la ar al conjunto de operaciones.
 - Elegir las operaciones que realizarán los objetos del nuevo tipo.
 - Implementación de los mensajes.
- Creación de objetos: los objetos se crean instanciando clases.
- Interfaz de un objeto: conjunto de todas las signaturas (nombre, parámetros y retorno de una operación declarada por el objeto) públicas definidas por él.
- Tipo: nombre utilizado para denotar una determinada interfaz. Un objeto puede tener muchos tipos, así como objetos muy diferentes pueden compartir un mismo tipo.
- Enlace dinámico: permite escribir programas que esperen un objeto con una interfaz determinada, sabiendo que cualquier objeto que implemente la interfaz aceptará la petición y sustituir en tiempo de ejecución un objeto por otro con misma interfaz (Polimorfismo).
- Polimorfismo:
 - Capacidad de dos o más tipos de objetos de responder al mismo mensaje, cada uno a su manera.
 - Un objeto no necesita saber a quién está enviando el mensaje: un objeto envía un mensaje, si el receptor implementa un método con la misma signatura responderá.

- Atributos:
 - Los objetos necesitan alguna forma de guardar información para su posterior uso por otros mensajes. Dicha información se guarda en los atributos.
 - Son utilizados exclusivamente para dar soporte a la implementación de los mensajes.
 - Probablemente cada vez que se cambie la forma de implementación, los atributos también cambien.
- Opciones y operandos:
 - En la implementación de una operación se utilizan:
 - Opción: representa una forma de operar con los operandos.
 - Operando: objeto que el método necesita para operar.
 - Cómo diferenciarlos:
 - Operandos permanecen estables mientras que las opciones pueden variar a medida que la clase evolucione.
 - Un parámetro es una opción si se puede encontrar un valor predeterminado adecuado.
 - Los argumentos de un mensaje deben ser sólo operandos.
- Estado de un objeto:
 - Combinación de los valores de los atributos de un objeto.
 - Los valores de los atributos determinarán el comportamiento del objeto ante los distintos mensajes que reciba.
 - Determina la respuesta del objeto ante cada uno de los mensajes: determina qué mensajes no son permitidos y el modo de actuar de los permitidos.
 - Guardar el estado de un objeto supone guardar la suficiente información para que al recuperarla se obtengan exactamente las mismas respuestas ante la misma secuencia de mensajes tanto del original como del recuperado.
 - Generalmente basta con guardar los atributos del objeto.
- Métodos de acceso:
 - **Encapsulación**: generalmente no se debe dar acceso a los datos internos de un objeto:


```
private int edad;
int getEdad() { ... } void setEdad(int edad) { ... }
```
 - Permite controlar las modificaciones del atributo:


```
void setEdad(int edad) {
    if (edad<0 || edad>150)
        throw new IllegalArgumentException... }
```
 - Ocultar los atributos permite modificar la implementación sin afectar a los clientes.
- Criterio de independencia:
 - Es una de las ventajas fundamentales de la orientación a objetos: poder trabajar con objetos con independencia de su implementación.
 - Lo único importante de un objeto son los mensajes que puede recibir.

2. Interfaces y clases abstractas:

- **Interfaces:** permiten comunicar objetos que no se conocen.
Si un jefe necesita delegar en un empleado para realizar una tarea que requiere especialización, hay varios empleados capaces de realizarla, pero no puede ser cualquiera pues necesita saber hacer dicha tarea:

```
public void contratar(EmpleadoInformatico empleado) {
    empleado.diseña();
}
interface EmpleadoInformatico {
    void diseña();
}
class Britanico implements EmpleadoInformatico {
    public void diseña() {
        Print(Diseñando);
    }
}
EmpleadoInformatico e = new Britanico();
```
- **Tipos de herencia:**
 - De interfaz: declaramos el compromiso de implementar unos métodos.
 - De implementación: creación de una clase a partir de la implementación de otra.
- Criterio de diseño: cuando un objeto debe delegar ciertas operaciones en otro deberá exigírselas mediante una interfaz.
- **Clases abstractas:** su principal uso es factorizar código propio.
 - Clase base abstracta: quedan mensajes sin implementar: lo que no es común. Se usa como implementación base para crear otras clases.
 - Una clase base puede dar implementación a todos los mensajes de la interfaz.
 - Su segundo uso principal es facilitar la extensión: son proporcionadas por el autor de la biblioteca o el framework de que se trate.
 - Se obtienen de implementar interfaces o clases concretas mediante refactoring.
- Uso de clases abstractas:
 - Si se desea ampliar la implementación heredada:

```
void trabaja() {
    // ampliación
    super.trabaja();
    // ampliación
}
```
 - Si se desea dar partes de la implementación de una operación:

```
interface Cantaor {
    void canta();
}
abstract class AbstractCantaor implements Cantaor {
    public void canta() {
        System.out.println("Carraspea");
        cuandoQuieraMaestro();
        System.out.println("Esquiva tomates");
    }
    protected abstract void cuandoQuieraMaestro();
}
class Jubilado extends AbstractCantaor {
    void cuandoQuieraMaestro() {
        System.out.println(" Soy mineroooo000ooo...");
    }
}
```

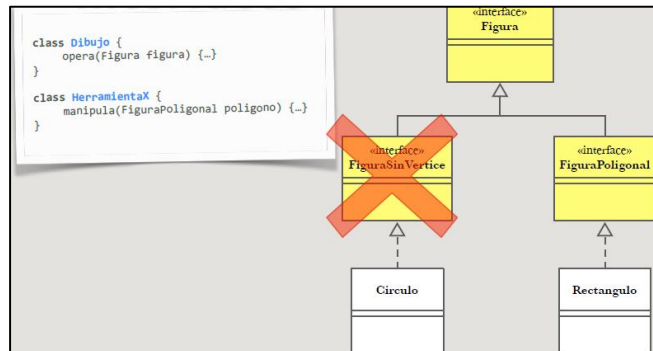
```

class Catedrático extends AbstractCantaor {
    void cuandoQuieraMaestro() {
        System.out.println("Relej, no marques las"); }
}
Catedrático donEnrique = new Catedrático();
donEnrique.canta();

```

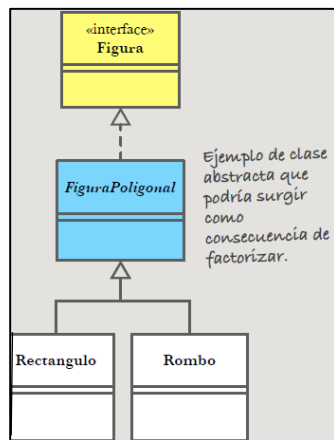
- Jerarquías de interfaces:

- No son necesarias para que el diseño sea orientado a objetos. Generalmente se obtiene un mal diseño.
- Las interfaces surgen por necesidad de declarar responsabilidades, nunca debe surgir una jerarquía como mera clasificación.



- Jerarquías de clases abstractas:

- Si aparece una nueva clase *Rombo* que implementa *Figura* y suponemos que *Rectángulo* y *Rombo* tienen código duplicado: movemos dicha funcionalidad común creando una nueva clase abstracta.



Una jerarquía de implementación de clases abstractas no es esencial que sea estable: el cliente solo conoce la interfaz, desconociendo lo que hay debajo de ella.

Las jerarquías de implementación son para compartir código común y puede reestructurarse cuantas veces se necesite:

Si por ejemplo desaparece *Rombo*, fusionar *FiguraPoligonal* y *Rectángulo*.

- **CONCLUSIONES:**

	Interfaz	Clase abstracta
Situación en que se originan	Extracción responsabilidades	Facilitar nuevas implementaciones. No repetir código
Creador	Diseñador	Programador
¿Raíz?	Si	No
Importancia	Fundamental: enlace dinámico	Comodidad
Operaciones	Añaden a las de sus ancestros	Implementan las de sus ancestros
Importancia cambios	Alto	Bajo

3. Principios de diseño Orientado a objetos:

- **Síntomas de un diseño pobre:**

- Rigidez: representa una resistencia al cambio. Un diseño es rígido si un simple cambio causa una sucesión de cambios en cascada en otros módulos independientes.
- Fragilidad: un programa es frágil si un cambio en una parte del sistema causa un fallo en otras no relacionadas conceptualmente.
- Inmovilidad: es difícil separar el código en componentes que puedan ser reutilizados en otros sistemas.
- Viscosidad: se refiere a que es más fácil hacer las cosas mal en lugar de bien. Distinguimos:
 - Viscosidad del software: si hacer los cambios respetando el diseño original es más costoso que hacerlos mal.
 - Viscosidad del entorno de desarrollo: si es lento e ineficiente.
- Complejidad innecesaria: si el diseño contiene elementos que no resultan útiles, cuando los desarrolladores pretenden adivinar demasiados cambios a los requisitos y se blindan ante ellos. No debemos caer en el **sobrediseño**: mucha funcionalidad no se utilizará y hace al software complejo.
- Repetición innecesaria: duplicación de código.
- Opacidad: dificultad de comprender un módulo o programa. Debemos mantener siempre el código tan claro y expresivo como sea posible: **que revele su intención**.

- **Principios de diseño Orientado a objetos:**

- **SRP --> Principio de responsabilidad única:**

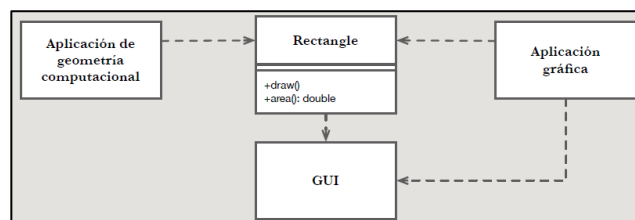
Una clase debería tener un único motivo de cambio. Así, definimos responsabilidad como una razón para el cambio: si podemos pensar en más de un motivo por el que la clase debería cambiar, dicha clase tiene más de una responsabilidad.

Ejemplo: dos aplicaciones distintas

- Geometría computacional (no dibuja un rectángulo)
- Aplicación gráfica (requiere igu o gui)

utilizan una clase Rectangle que tiene dos métodos: uno dibuja y otro calcula el área.

Este diseño viola el principio pues la clase Rectangle tiene dos responsabilidades: sería necesario incluir una igu y si una de las dos aplicaciones requiriese un cambio en la clase, afectaría a la otra.



- **OCP --> Principio de abierto-cerrado:**

Las entidades software (clases, módulos...) deberían estar abiertas para la extensión, pero cerradas a la modificación. Cuando un simple cambio a un programa resulta en una cascada de cambios, el diseño adolece de rigidez. La idea es que los cambios se hagan añadiendo código nuevo, no modificando el existente.

Ejemplo Cliente – Servidor:

Cliente -----> Servidor *El cliente usa el servidor per si Servidor
requiere un cambio, habría que cambiar Cliente.*

Cliente -----> <interface> <----- ServidorImpl
Servidor

Así Cliente estaría abierto y cerrado.

○ **LSP --> Principio de sustitución de Liskov:**

Los objetos de un programa deberían ser reemplazables por instancias de sus subtipos sin alterar el correcto funcionamiento del programa. Ó: debemos asegurarnos de que las clases derivadas extiendan la clase base sin alterar su comportamiento.

Problema clásico: Rectángulo <----- Cuadrado

○ **DIP --> Principio de inversión de dependencias:**

Hay que depender de abstracciones, no de implementaciones correctas (Interfaces).

○ **ISP --> Principio de segregación de interfaces:**

Es mejor muchas interfaces específicas para cada cliente que una sola interfaz de propósito general. Los clientes no deberían depender de métodos que no usan.

Ejemplo:

```
public interface Database {  
    void add();  
}  
public class Customer implements Database {  
    ...  
    public void add() {...}  
}
```

Si ahora algunos clientes desean leer de la bbdd... en lugar de añadir el método read a la interfaz Database, **creamos una nueva interfaz** DatabaseV1:

```
public interface DatabaseV1 {  
    void read();  
}  
public class CustomerWithRead implements Database, DatabaseV1 {  
    public void read() {...}  
    public void add() {...}  
}
```

```
Database o = new Customer(); // los antiguos  
o.add();
```

```
DatabaseV1 o1 = new CustomerWithRead(); // nuevos clientes  
o1.read();
```

• **Principios GRASP:**

- Son una serie de principios generales para asignar responsabilidades.
- **Responsabilidad:** se refiere a las obligaciones de un objeto, definen su comportamiento.
 - Entre las responsabilidades de “**hacer**” se encuentran:
 - Hacer algo con él mismo.
 - Iniciar una acción en otros objetos.
 - Controlar y coordinar actividades en otros objetos.
 - Entre las responsabilidades de “**conocer**” se encuentran:
 - Conocer datos privados encapsulados y objetos relacionados.

GRASP nº1. Experto en información:

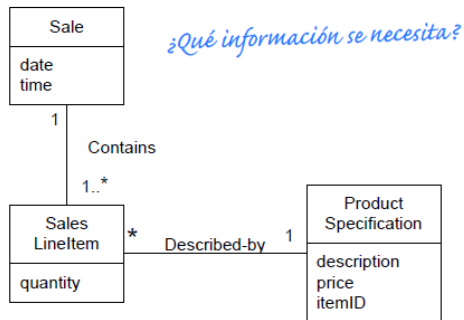
Asignar una responsabilidad al experto: clase que tiene la información necesaria para llevarla a cabo.

Dónde mirar qué clase tiene la información necesaria:

Si tenemos un modelo de diseño detallado y hay clases relevantes, miraremos ahí.

Si no, en el de dominio.

Ejemplo: asignar responsabilidad de calcular el importe total de la venta:



Venta (sale) es quien contiene las líneas de venta y habría que asignarle a ella la responsabilidad getTotal().

Se asignaron tres responsabilidades a tres clases del diseño: venta (conoce el total), línea de venta (conoce el subtotal de la línea) y producto (conoce el total del producto).

Contraindicaciones: si la clase venta tuviera que contener la lógica relacionada con la gestión de bbdd: tendría otra responsabilidad más que disminuye la cohesión, dependería de clases de servicio de otro subsistema (está acoplada).

Ventajas: se mantiene el encapsulamiento de la información y se distribuye el comportamiento entre las clases que contienen la información requerida.

GRASP nº2. Creador:

Asignar a la clase B la responsabilidad de crear una instancia de A si se cumple alguna de estas condiciones:

- B agrega objetos de A.
- B contiene objetos de A.
- B registra objetos de A.
- B utiliza más estrechamente objetos de A.
- B tiene datos de inicialización de un objeto de A.

Ejemplo: en el mismo caso que el 'experto en información' ¿quién debería ser el responsable de crear las líneas de venta? Es conveniente delegar la creación a una clase de fabricación auxiliar.

GRASP nº3. Bajo acoplamiento:

Cómo lograr que los cambios tengan un bajo impacto e incrementar la reutilización -->

Asignar una responsabilidad de manera que el acoplamiento permanezca bajo.

El **acoplamiento** es una medida de la fuerza con que un elemento está conectado a otros. Si una clase presenta un fuerte acoplamiento con otras:

- Cambios en esas clases obligan a cambios en ella.
- Difícil de entender de manera aislada.
- Difícil de reutilizar.

Formas comunes de acoplamiento:

- TipoX tiene un atributo que hace referencia a una instancia de TipoY.
- Un objeto de TipoX invoca servicios de un objeto TipoY.
- TipoX tiene un método que hace referencia a una instancia de TipoY, o al propio TipoY.
- TipoX es una subclase, directa o indirecta, del TipoY.
- TipoY es una interfaz y el TipoX implementa esa interfaz.

Cuestiones importantes:

- El bajo acoplamiento promueve clases más independientes entre sí.
- Las subclases están fuertemente acopladas a sus superclases: las ObjP de la que heredan las que hay que guardar en una bdd --> acopla altamente objetos del dominio a una determinada tecnología.
- Cuanto más genéricas y con más probabilidad de reutilización son las clases, más bajo debería ser el acoplamiento.
- Caso extremo de bajo acoplamiento --> no exista acoplamiento entre clases.
- Alto acoplamiento es un problema entre elementos que no son estables.

Ventajas:

- No afectan a cambios en otros componentes.
- Fácil de entender de manera aislada.
- Favorece reutilización.

GRASP nº4. Alta cohesión:

Asignar una responsabilidad de manera que la cohesión permanezca alta.

Cohesión:

- Mide el grado de conectividad entre las funciones y elementos de un mismo módulo. La mejor es la cohesión funcional.
- Un elemento con responsabilidades altamente relacionadas tiene una alta cohesión.
- Los elementos pueden ser clases, módulos, sistemas, subsistemas...
- Una clase con baja cohesión hace muchas cosas no relacionadas.
- Problemas: difíciles de entender, de reutilizar, de mantener.

Diseño modular: la **modularidad** es la propiedad de un sistema que se ha descompuesto en un conjunto de módulos cohesivos y débilmente acoplados.

Suficiencia, completitud y ser primitivo:

- Suficiente: el componente representa suficientes características de una abstracción como para permitir una abstracción significativa con el componente.
- Completo: la interfaz del componente representa todas las características de la abstracción.
- Primitivo: si todas las operaciones que realiza un componente pueden implementarlas fácil y eficientemente.

GRASP nº5. Controlador:

Asignar la responsabilidad de recibir o manejar un evento a una clase que representa una de las siguientes opciones:

- Representa el sistema global, un dispositivo o un subsistema.
- Representa un escenario de caso de uso en el que tiene lugar el evento.

GRASP nº6. Polimorfismo:

Cuando las alternativas o comportamientos relacionados varían según el tipo del objeto, se asignará la responsabilidad a los tipos para los que varía el comportamiento, empleando operaciones polimórficas.

Hay que intentar huir de la lógica condicional: if anidados, instanceof, es tan...

GRASP nº7. Fabricación pura:

¿Qué objetos deberían tener la responsabilidad cuando no se quiere violar los principios de alta cohesión y bajo acoplamiento, pero, sin embargo, las soluciones que ofrece el experto no son adecuadas?

Se asignará un conjunto de responsabilidades altamente cohesivo a una clase artificial, de conveniencia, que no representa un concepto del dominio del problema, sino que se ha inventado para permitir esa alta cohesión, bajo acoplamiento y la reutilización de código.

Ejemplo: guardar ventas en una bbdd: la clase perdería cohesión, debe acoplarse a la bbdd.

GRASP nº8. Indirección:

Dónde asignar una responsabilidad para evitar acoplamiento --> a un objeto intermedio que medie entre los componentes de manera que no se acoplen directamente.

GRASP nº9. Variaciones protegidas:

Cómo diseñar objetos de manera que las variaciones no tengan impacto en otros elementos --> identifique los puntos de variaciones previstas, asigne responsabilidades para crear una interfaz estable alrededor de ellos.

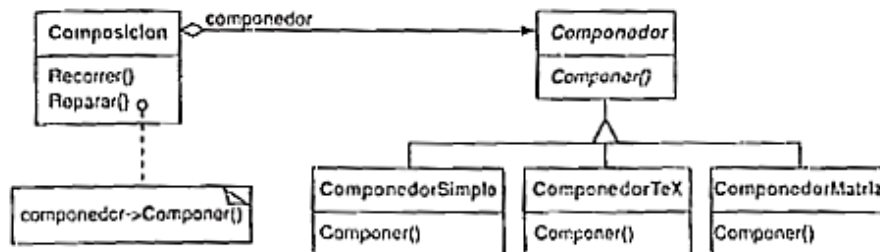
DISEÑO DEL SOFTWARE – TEORÍA PATRONES

FALTAN LOS PATRONES VISITOR, PROTOTYPE Y FACTORY METHOD

5A. Strategy:

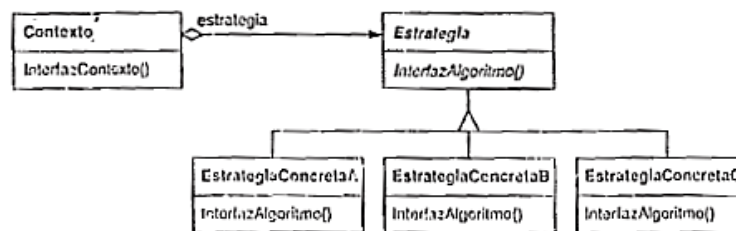
- **Propósito:** define una familia de algoritmos, **encapsula** cada uno de ellos y los hace intercambiables. Permite que un algoritmo varíe independientemente de los clientes que lo usen.
- **Motivación:** existen varios algoritmos para dividir en líneas un flujo de texto. Codificar dichos algoritmos en las clases que los usan no resulta una buena práctica por:
 - Los clientes que necesitan dividir el texto se vuelven más complejos si tienen que incluir dicho código.
 - Los distintos algoritmos serán apropiados en distintos momentos. No se debe permitir múltiples algoritmos si no se van a utilizar todos.
 - Es difícil añadir nuevos algoritmos o modificar los existentes cuando la división en líneas es parte integral de un cliente.

Estos problemas se evitan definiendo clases que encapsulen los diferentes algoritmos de división de líneas. Un algoritmo así encapsulado se denomina estrategia.



La clase *Composición* debe mantener y actualizar los saltos de línea. Las estrategias de división no están implementadas en esa clase. Se implementan separadas por las subclases de la interfaz **Compondor**.

- **Estructura y participantes:**



- **Estrategia (compondor):** declara una interfaz común.
- **EstrategiaConcretaX (simple, tex, matriz):** implementa los distintos algoritmos.
- **Contexto (composición):** se configura como una estrategia concreta. Mantiene una referencia a un objeto estrategia. Puede definir una interfaz que permita a la Estrategia acceder a sus datos.
- **Colaboraciones:**
 - Estrategia y contexto colaboran para implementar el algoritmo escogido: el contexto puede pasar todos los datos que necesita al llamar a la estrategia concreta.
 - Los clientes colaboran con el contexto: pueden pasarle la estrategia concreta.

- Consecuencias:

- Define familias de algoritmos relacionados.
- Es una alternativa a la herencia:
 - Hace que el contexto sea más fácil de entender, modificar y mantener.
 - Evita duplicación de código.
 - Evita explosión de subclases.
 - Se puede cambiar dinámicamente.
- Elimina múltiples sentencias condicionales:

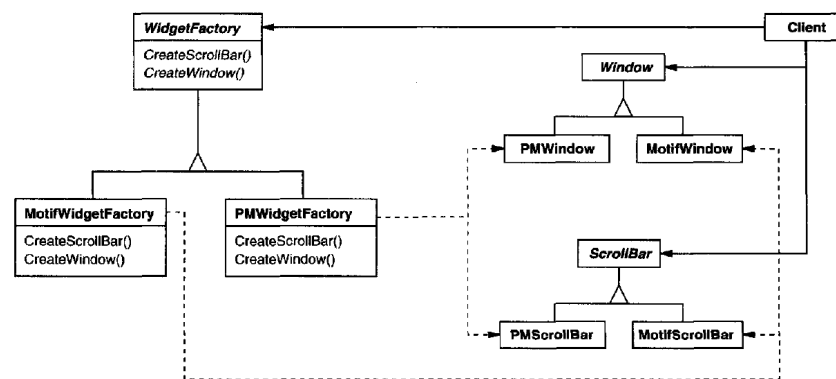
```
void Composition::Repair () {
    switch(_breakingStrategy) {
        case SimpleStrategy:
            ComposeWithSimpleCompositor();
            break;
        case TeXStrategy:
            ComposeWithTeXCompositor();
            break;
        // ...
    }
}
```

- El cliente puede elegir entre varias implementaciones.
- Los clientes deben conocer las diferentes estrategias. Deben comprender cómo difieren las estrategias antes de seleccionar la adecuada.
- Puede complicarse la comunicación entre contexto y estrategias.
- Crece el número de objetos.

5B. Abstract Factory:

- **Propósito:** proporciona una interfaz para crear familias de objetos relacionados o que dependen entre sí, sin especificar sus clases concretas.
- **Motivación:** Pensemos en un toolkit de interfaces de usuario que admita múltiples estándares de interfaces de usuario. Los distintos estándares definen distintos aspectos y formas de comportamiento de los “útiles” de la interfaz de usuario. Para que una aplicación pueda portarse a varios estándares de interfaz de usuario, esta no debería codificar sus útiles para una interfaz de usuario en particular. Si la aplicación crea instancias de clases o útiles específicos de la interfaz de usuario será difícil cambiar esta más tarde.

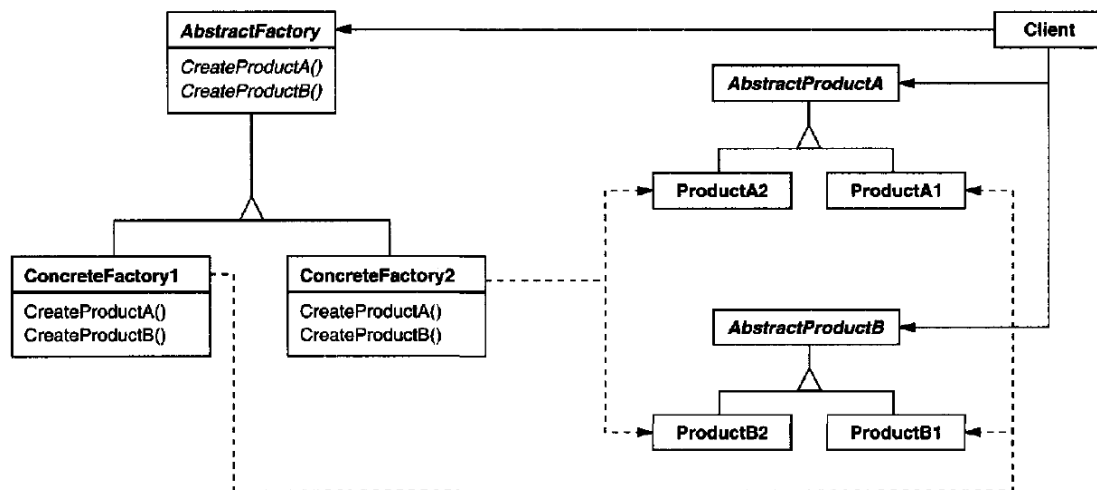
Se puede solucionar este problema definiendo una clase abstracta *FabricaDeUtiles*, que declara una interfaz para crear cada tipo básico de útil (widget). También hay una clase abstracta para cada tipo de útil y las subclases concretas implementan útiles para un estándar concreto de interfaz de usuario. LA interfaz de *FabricaDeUtiles* tiene una operación que devuelve un nuevo objeto para cada clase abstracta de útil. Los clientes llaman a estas operaciones para obtener instancias de útiles, pero no saben que clases concretas utilizan (clientes independientes de la interfaz de usuario).



Hay una subclase *FabricaDeUtiles* para cada estándar de interfaz de usuario. Cada subclase implementa operaciones que crean el útil apropiado para su interfaz de usuario. Los clientes crean útiles únicamente a través de la interfaz *FabricaDeUtiles* y no tienen conocimiento de las clases que implementan los útiles para una determinada interfaz de usuario (no se tienen que atar a una clase concreta, sino a una sola interfaz definida por una clase abstracta).

Una *FabricaDeUtiles* también fuerza a que se cumplan las dependencias entre las clases concretas de útiles. Si queremos una barra Motif, solo podremos usar elementos Motif debido al uso de la clase *FabricaDeUtilesMotif*.

- **Aplicabilidad:** Se usa el patrón Abstract Factory cuando:
 - Un sistema debe ser independiente de cómo se crean, componen y representan sus productos.
 - Un sistema debe de ser configurado con una entre múltiples familias de productos.
 - Una familia de objetos producto relacionados está diseñada para ser usada conjuntamente, y es necesario hacer cumplir esta restricción.
 - Quiere proporcionar una biblioteca de clases de productos, y solo quiere revelar sus interfaces, no sus implementaciones.
- **Estructura y participantes:**



- **FabricaAbstracta (FabricaDeUtiles):** declara una interfaz para operaciones que crean objetos producto abstracto.
 - **FabricaConcreta (FabricaDeUtilesMotif, FabricaDeUtilesPM):** implementa las operaciones para crear objetos producto concretos.
 - **ProductoAbstracto (Ventana, BarraDeDesplazamiento):** declara una interfaz para un tipo de objeto producto.
 - **ProductoConcreto (VentanaMotif, BarraDeDesplazamientoMotif):** define un objeto producto para que sea creado por la fabrica correspondiente. Implementa la interfaz ProductoAbstracto.
 - **Cliente:** solo usa interfaces declaradas por las clases FabricaAbstracta y ProductoAbstracto.
- Colaboraciones:
 - Normalmente solo se crea una única instancia de una clase FabricaConcreta en tiempo de ejecución. Esta fabrica crea objetos producto que tienen una determinada implementación. Para crear diferentes objetos producto, los clientes deben usar una fábrica concreta diferente.
 - FabricaAbstracta delega la creación de objetos producto en su subclase FabricaConcreta.
 - Consecuencias:

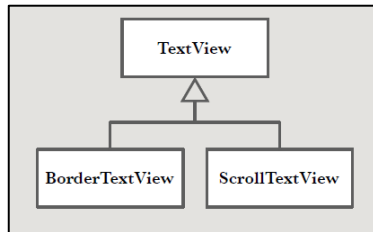
El patrón Abstract Factory tiene las siguientes ventajas e inconvenientes:

 - 1- Aísla las clases concretas. Este patrón ayuda a controlar las clases de objetos que crea una app. Como una fábrica encapsula la responsabilidad y el proceso de creación de un objeto, aísla a los clientes de las clases de implementación. Los clientes manipulan las instancias a través de sus interfaces abstractas. Los nombres de las clases producto quedan aisladas en la implementación de la fábrica concreta; no aparecen en el código cliente.
 - 2- Facilita el intercambio de familias de productos. La clase de una fábrica concreta solo aparece una vez en una aplicación (cuando se crea). Esto facilita cambiar la fábrica concreta que usa una aplicación. Como una fábrica abstracta crea una familia completa de productos, toda la familia de productos cambia de una vez (podemos cambiar de Motif a PM solo cambiando los objetos fabrica).

- 3- Promueve la consistencia entre productos. Cuando se diseñan objetos producto en una familia para trabajar juntos, es importante que una app use objetos de una sola familia a la vez (FabricaAbstracta facilita la restricción).
- 4- Es difícil dar cabida a nuevos tipos de productos. Ampliar las fabricas abstractas para producir nuevos tipos de productos no es fácil. Esto se debe a q la interfaz FabricaAbstracta fija el conjunto de productos que se pueden crear. Permitir nuevos tipos de productos requiere ampliar la interfaz de fábrica, lo que a su vez implica cambiar la clase FabricaAbstracta y todas sus subclases.

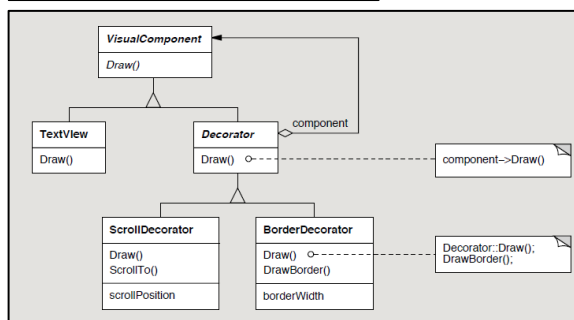
6. Decorator:

- **Propósito:** añade responsabilidades adicionales a un objeto dinámicamente. Los decoradores proporcionan una alternativa flexible a la herencia para extender funcionalidad.
- **Motivación:** a veces queremos añadir responsabilidades a objetos individuales, no a toda una clase. ¿Cómo añadir un borde o barra de desplazamiento a un componente de interfaz de usuario?



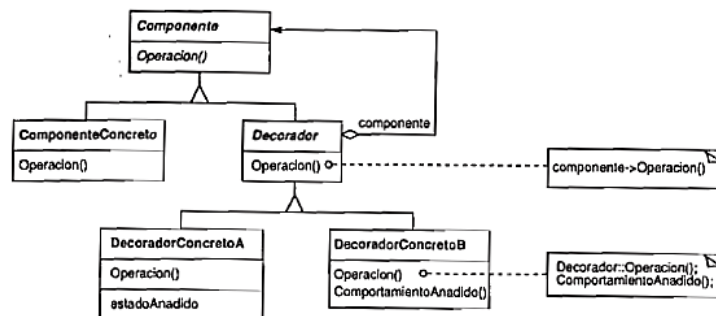
Si utilizamos herencia, la elección del borde se hace estáticamente, no la puede hacer el cliente. No podríamos tener un TextView con borde y barra de desplazamiento.

Solución: **envolver el componente en otro objeto que sea quien añada el borde: Decorador.**



El decorador sigue cumpliendo la interfaz del objeto original, su presencia es transparente para los clientes del componente.

- **Aplicabilidad:**
 - Para añadir responsabilidades a otros objetos dinámicamente y de forma transparente.
 - Cuando no se puede heredar o no resulta práctico.
- **Estructura y participantes:**



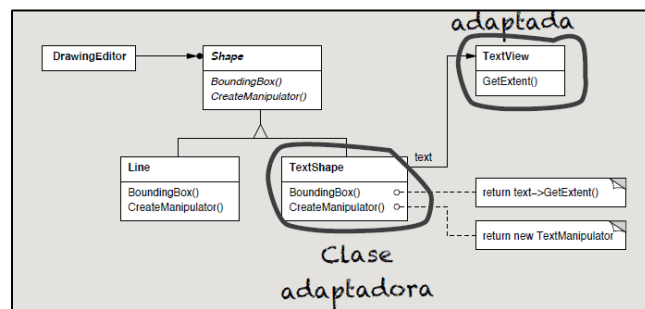
- **Componente (VisualComponent):** define la interfaz para objetos a los que se puede añadir responsabilidades dinámicamente.
 - **ComponenteConcreto (TextView):** define un objeto al que se pueden añadir responsabilidades adicionales.
 - **Decorator:** mantiene una referencia a un objeto Componente y define una interfaz que se ajusta a la interfaz del Componente.
 - **DecoratorConcreto:** añade responsabilidades al componente.
- **Consecuencias - ventajas:**
 - Más flexibilidad que la herencia estática.
 - Evita que las clases de arriba de la jerarquía estén repletas de funcionalidades.

- Consecuencias – inconvenientes:
 - Un decorador y sus componentes no son idénticos.
 - Muchos objetos pequeños: sistema más difícil de aprender y depura.
- Patrones relacionados:
 - Adapter: el decorador sólo cambia las responsabilidades del objeto, no su interfaz.
 - Composite: un decorador puede verse como un composite de un solo componente, pero añade responsabilidades adicionales. No está pensado para la agregación de objetos.
 - Strategy: el decorador amplía la piel del objeto, una estrategia cambia sus tripas.
- Aspectos clave:
 - Los decoradores tienen mismo tipo que los objetos que decoran.
 - Se puede usar uno o más decoradores para envolver un objeto.
 - El decorador añade su propio comportamiento antes o después de delegar al objeto decorado el resto del trabajo.

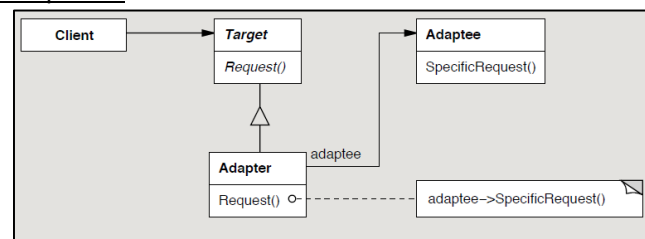
7A. Adapter:

- **Propósito:** convierte una clase en otra interfaz que es la que esperan los clientes. Permite que cooperen clases que de otra forma no podrían por tener interfaces incompatibles.
- **Motivación:** suponiendo que diseñamos un editor de dibujo:
 - La abstracción es fundamental es el objeto gráfico (Shape) que puede dibujarse a sí mismo.
 - Define una subclase para cada tipo de objeto gráfico: LineShape, PolygonShape.Suponiendo que para implementar una clase TextShape queremos utilizar una clase TextView que nos proporciona la biblioteca gráfica:

Solución: **creamos una clase TextShape que adapte la interfaz de TextView a la de Shape.**



- **Aplicabilidad:**
 - Queremos usar una clase existente y esta no tiene la interfaz que necesitamos.
 - Queremos crear una clase reutilizable que coopere con las que no está relacionada.
 - Necesitamos usar varias subclases existentes, pero sin tener que adaptar su interfaz.
- **Estructura y participantes:**



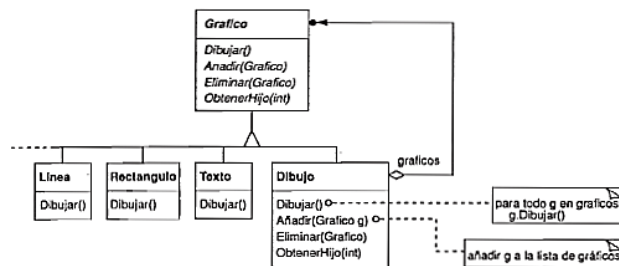
- **Target** (Shape): define la interfaz específica del dominio.
- **Client** (DrawingEditor): colabora con objetos de tipo target.
- **Adaptee** (TextView): clase existente cuya interfaz necesita ser adaptada.
- **Adapter** (TextShape): adapta la interfaz de Adaptee a la de Target.
- **Consecuencias:**
 - Las versiones de clases y objetos de este patrón tienen diferentes ventajas e inconvenientes.
 - **Un adaptador de clases:**
 - Adapta una clase concreta a una interfaz.
 - Permite que el adaptador redefina parte del comportamiento de la clase adaptada. Introduce un solo objeto adicional.

- Un adaptador de objetos:
 - Permite que un único adaptador funcione con objetos de cualquiera de sus subclases.
 - No es del tipo objeto adaptado.

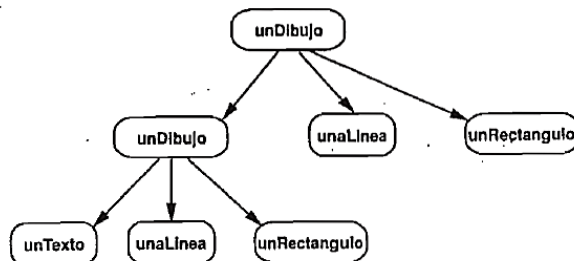
7B. Composite:

- **Propósito:** permite componer objetos en estructuras arbóreas para representar jerarquías de todo-parte, de modo que los clientes puedan tratar a los objetos individuales y a los compuestos de manera uniforme.
- **Motivación:** un editor de dibujo permite realizar dibujos compuestos de elementos simples (líneas, rectángulos...) u otros dibujos. El usuario puede agrupar componentes para formar componentes más grandes, que a su vez pueden agruparse para formar componentes aún mayores. Cómo evitar que los clientes tengan que distinguir entre unos y otros.

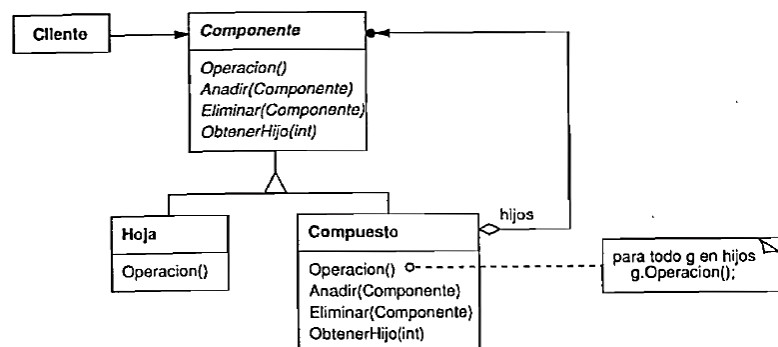
La clave del patrón Composite es una clase abstracta que representa tanto a primitivas como a sus contenedores.



- En el ejemplo la **clase abstracta es la clase Gráfico**, que declara operaciones como Dibujar que son específicas de objetos gráficos.
- Las **subclases Línea, Rectángulo... definen objetos gráficos primitivos**.
- La clase **Dibujo define una agregación de objetos Gráfico**. Implementa Dibujar() para que llame al Dibujar() de sus hijos, y añade operaciones realizadas con los hijos.



- **Aplicabilidad:**
 - Representar jerarquías de parte-todo.
 - Que los clientes traten por igual los objetos individuales y los compuestos.
- **Estructura y participantes:**



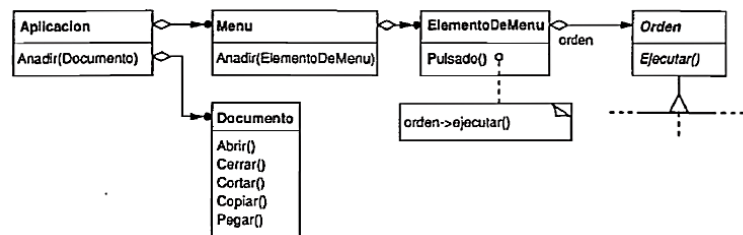
- Componente (Gráfico):
 - Declara la interfaz de los objetos de la composición.
 - Implementa el comportamiento predeterminado de la interfaz que es común a todas las clases.
 - Declara una interfaz para acceder a sus componentes hijos y gestionarlos.
 - **Puede proporcionar implementaciones predeterminadas que luego las clases hoja y compuestas redefinan.** Puede haber operaciones que tengan sentido en unas, pero en otras no.
- Hoja (Rectángulo, Línea...):
 - Representa los objetos hoja de la composición. Una hoja no tiene hijos.
 - Define el comportamiento de los objetos primitivos de la composición.
- Compuesto (Dibujo):
 - Define el comportamiento de los componentes que tienen hijos.
 - Almacena componentes hijos.
 - Implementa las operaciones de la interfaz Componente relacionadas con los hijos.
- Cliente: manipula los objetos de la composición a través de Componente.
- Consecuencias:
 - Permite jerarquías de objetos tan complejas como se quiera.
 - Simplifica el cliente al eliminar código para distinguir entre unos y otros.
 - Se pueden añadir nuevos componentes fácilmente.
 - Como desventaja podría hacer el diseño demasiado general si queremos restringir componentes que puedan formar parte de un compuesto determinado.

8A. Command:

- **Propósito:** encapsula una petición dentro de un objeto, permitiendo parametrizar a los clientes con distintas peticiones, encolarlas, guardarlas en un registro de sucesos o implementar un mecanismo de deshacer/repetir. También conocido como Action.
- **Motivación:** una biblioteca de clases para interfaces de usuario tendrá objetos como botones y elementos de menú responsables de realizar alguna operación. La biblioteca no puede implementar dichas operaciones directamente en el botón o el menú: sólo las aplicaciones que usan la biblioteca saben qué hacer y a qué operaciones de otros objetos hay que llamar.

La clave de este patrón es una interfaz **Command que define una operación **Execute**.**

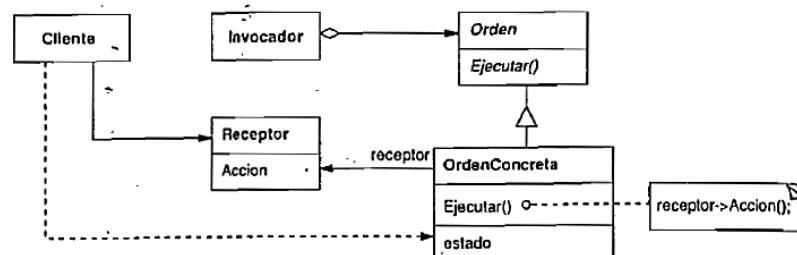
Son las subclases concretas quienes implementan la operación y especifican el receptor de la orden.



Podemos configurar cada elemento del menú con un objeto Command. Los elementos del menú no saben qué objeto concreto están utilizando.

Lo que permite este patrón es desacoplar al objeto que invoca a la operación de aquel que tiene el conocimiento necesario para realizarla.

- **Aplicabilidad:**
 - Úsese este patrón cuando se quiera parametrizar objetos con una determinada acción.
 - Que la acción a realizar y el objeto que lanza la petición tenga ciclos de vida distintos.
 - **Permitir deshacer o repetir:** en este caso, **execute** deberá guardar el estado para poder revertir los efectos de ejecutar la operación. Hará falta una operación **unexecute**.
 - Guardar todas las operaciones ejecutadas en un registro log.
 - Usar transacciones.
- **Estructura y participantes:**

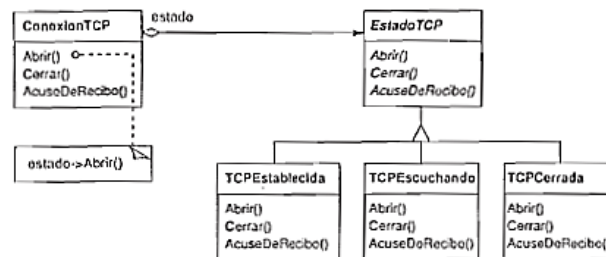


- **Orden:** declara una interfaz para ejecutar una operación.
- **OrdenConcreta (OrdenPagar, OrdenAbrir):**
 - Define un enlace entre un objeto Receptor y una acción.
 - Implementa **Ejecutar** invocando la correspondiente operación(es) del Receptor.
- **Cliente (Aplicación):** crea un objeto **OrdenConcreta** y establece su receptor.

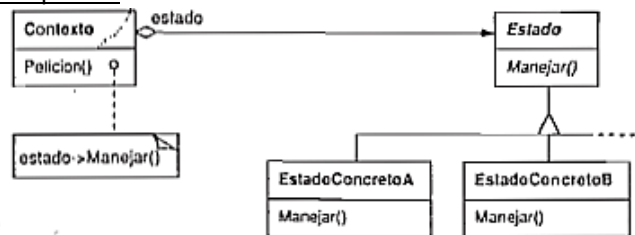
- Invocador (ElementoDeMenú): pide a la orden que ejecute la petición.
- Receptor (Documento, Aplicación): sabe cómo llevar a cabo operaciones asociadas a una petición.
- Colaboraciones:
 - El cliente crea un objeto OrdenConcreta y especifica su receptor.
 - Un objeto Invoker guarda el objeto OrdenConcreta.
 - Aquella llama a la operación de este último.
 - El objeto OrdenConcreta se vale de operaciones de su receptor para llevar a cabo la acción.
- Consecuencias:
 - Desacopla el objeto que llama a la operación del que sabe cómo llevarla a cabo.
 - Se puede ensamblar (Composite).
 - Resulta sencillo añadir nuevas acciones, al no tener que tocar las clases existentes.

8B. State:

- Propósito: permite a un objeto alterar su comportamiento cuando cambia su estado interno. Parecerá como si el objeto hubiera cambiado su clase.
- Motivación: una conexión de red es representada como ConexionTCP. La conexión puede estar en uno de los siguientes estados:
 - Abierta.
 - Escuchando.
 - Cerrada.



- Aplicabilidad:
 - El comportamiento de un objeto depende de su estado.
 - Las operaciones tienen sentencias condicionales anidadas que tratan con los estados (siendo el estado normalmente una constante).
- Estructura y participantes:

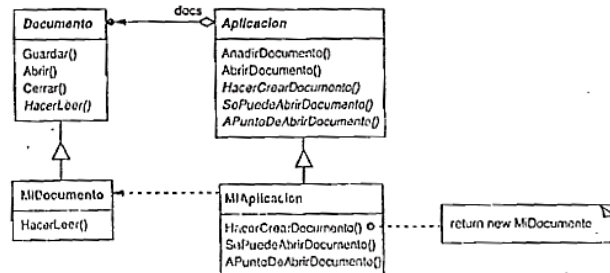


- Contexto (ConexionTCP):
 - Define la interfaz que interesa a los clientes.
 - Mantiene una referencia a una subclase de estado concreto que representa el estado actual.
- Estado (EstadoTCP): define la interfaz para encapsular el comportamiento asociado con el estado del contexto.
- Subclases EstadoConcretoX (TCPEstablecida, TCPEscuchando): cada subclase implementa las operaciones anteriores para ese estado concreto.
- Colaboraciones:
 - El Contexto delega las operaciones dependientes del estado al objeto que representa el estado actual.
 - El Contexto podría pasarse a sí mismo como parámetro.
 - Una vez el Contexto es inicializado en un determinado estado, los clientes no necesitan tratar directamente con los estados.
 - O bien el Contexto o bien los Estados Concretos deciden cuándo se cambia de estado.
- Consecuencias:
 - Localiza el comportamiento específico del estado y lo aísla en un objeto.
 - Hace explícitas las transiciones entre estados.

- Implementación:
 - Uso de herencia dinámica: este patrón no sería necesario en lenguajes que permitan cambiar la clase de un objeto en tiempo de ejecución.
 - Acciones de E/S: cuando hay que realizar alguna acción al entrar en un estado o salir de él puede venir bien añadir un método entry/exit a la interfaz del estado.

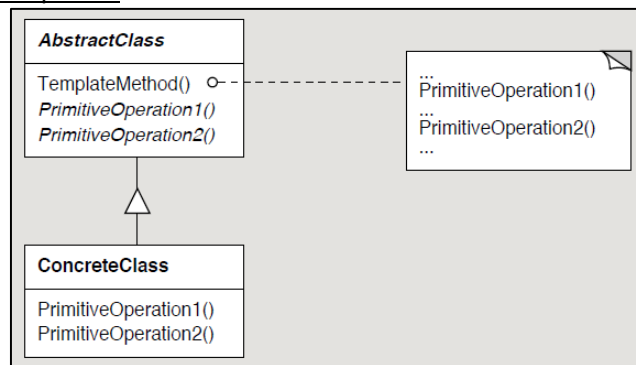
8C. Template:

- **Propósito:** define el esqueleto de un algoritmo en una operación, defiriendo algunos pasos hasta las subclases. Permite que estas redefinan ciertos pasos del algoritmo sin cambiar su estructura en sí.
- **Motivación:** sea un framework de aplicaciones que nos proporcionan las clases Aplicación (abre los documentos) y Documento (documento en sí). Las aplicaciones construidas con el framework redefinirán ambas clases para adaptarlas a sus necesidades concretas.



El método abstracto `AbrirDocumento()` define los pasos necesarios para abrir un documento. Lo hace en términos de operaciones abstractas: que las clases redefinirán convenientemente.

- **Aplicabilidad:**
 - Para implementar las partes de un algoritmo que no cambian y dejar que las subclases implementen aquellas que pueden variar.
 - Como motivo de factorizar código, cuando movemos cierto código a una clase base común para evitar código duplicado.
 - Para controlar el modo en que las subclases extienden de la clase base.
- **Estructura y participantes:**

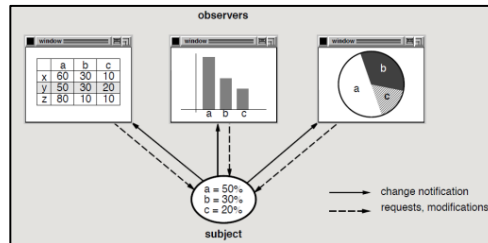


- **AbstractClass (Aplicación):**
 - Define las operaciones primitivas abstractas que redefinirán las subclases.
 - Implementa un método de plantilla con el esqueleto del algoritmo.
- **ConcreteClass (MiAplicación):**
 - Implementa las operaciones primitivas.
- **Consecuencias:**
 - Los métodos de plantilla son una técnica fundamental para la reutilización de código.
 - Inversión de control: es la clase padre quién llama a operaciones en los hijos.
 - Los métodos de plantilla pueden llamar a los siguientes tipos de operaciones:
 - O concretas de otra clase.
 - O concretas en la propia clase base abstracta.

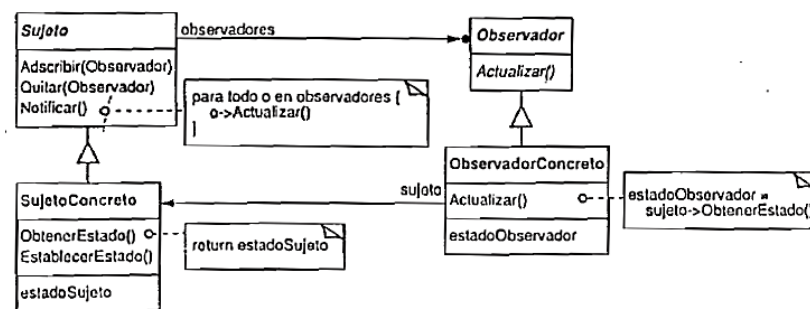
- Operaciones primitivas.
- Patrones relacionados:
 - Strategy: Template usa la herencia para **modificar parte de un algoritmo**. El Strategy usa delegación para **cambiar el algoritmo entero**.
 - Factory Method: Muchas veces los métodos de fabricación son llamados desde métodos de plantilla
- Operaciones “de enganche”:
 - Una subclase puede extender una operación de la clase padre llamando a la operación: **super**.operation();
- Implementación:
 - Hacer las operaciones primitivas llamadas por el método de plantilla **Protected**, y aquellas que deben ser obligatoriamente redefinidas, **Abstract. (*) en código**.
 - Minimizar el número de operaciones primitivas abstractas.

9A. Observer:

- Propósito: define una dependencia uno-a-muchos entre objetos, de modo que cuando un objeto cambia su estado, todos los demás objetos dependientes se modifican y actualizan automáticamente.
- Motivación:



- Aplicabilidad:
 - Una abstracción tiene dos aspectos, uno de los cuales depende de otro.
 - Un cambio en un objeto requiere cambios en otros.
 - Un objeto necesita notificar a otros cambios en su estado sin hacer presunciones sobre quienes son dichos objetos.
- Estructura y participantes:



- Sujeto: conoce a sus observadores. Un objeto puede ser observado por cualquier número de objetos Observador.
- Observador: define una interfaz para actualizar los objetos que deben ser notificados ante cambios en un sujeto.
- SujetoConcreto:
 - Almacena el estado de interés para objetos ObservadorConcreto.
 - Envía una notificación a sus observadores cuando cambia su estado.
- ObservadorConcreto:
 - Mantiene una referencia a un objeto SujetoConcreto.
 - Guarda un estado que debería ser consistente con el del sujeto.
 - Implementa la interfaz de actualización del Observer.
- Consecuencias:
 - Permite variar objetos observados y observadores independientemente:
 - Se puede reutilizar los objetos observados sin sus observadores y viceversa.
 - Se puede añadir nuevos observadores sin modificar ninguna de las clases existentes.
 - Acoplamiento abstracto entre Sujeto y Observador: todo lo que un objeto sabe sobre sus observadores es que tiene una lista de objetos que satisfacen la interfaz Observador.
 - No se especifica el receptor de una actualización.

DISEÑO DEL SOFTWARE – PRÁCTICAS

Práctica 1: Videoclub:

PATRÓN STATE

El **patrón State** permite a un objeto alterar su comportamiento cuando varía su estado interno. Parecerá como si el objeto hubiera cambiado de clase.

En la aplicación del videoclub:

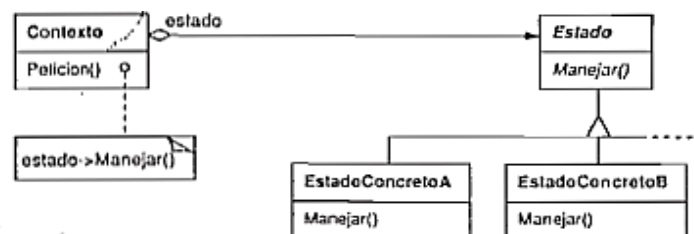
- El comportamiento de la clase Movie depende de su estado, o sea, de su categoría. En función de la categoría el precio y los puntos se calculan de forma diferente.
- Se elimina la lógica condicional para calcular el precio ya que las nuevas clases implementan la interfaz categoría.

```
// 1) Calcula el importe de cada alquiler
double price = 0;
switch (rental.getMovie().getCategory()) {
case Movie.NEW_RELEASE:
    price += rental.getDays() * 3;
    break;
case Movie.REGULAR:
    price += 2;
    if (rental.getDays() > 2)
        price += (rental.getDays() - 2) * 1.5;
    break;
case Movie.CHILDRENS:
    price += 1.5;
    if (rental.getDays() > 3)
        price += (rental.getDays() - 3) * 1.5;
    break;
}

// 2) Cada alquiler da 1 punto. Punto extra para novedades alquiladas
// 2 o más días
int points = 1;
if ((rental.getMovie().getCategory() == Movie.NEW_RELEASE) &&
    rental.getDays() > 1)
    points = 2;
```

- El estado de la película puede cambiar en tiempo dinámico.

Contexto:



- El **contexto sería la clase Movie**. Define la interfaz que interesa a los clientes.

```
public class Movie
{
    private String title;
    private Category category;
    public Movie(String title, Category category) {
```

```

        this.title = title;
        this.category = category;
    }

    public int getPoints(int days) {
        return category.getPoints(days);
    }

    public double getPrice(int days) {
        return category.getPrice(days);
    }

    // ...

```

- El estado sería la interfaz Category. Define la interfaz para encapsular el comportamiento asociado al estado del contexto.

```

public interface Category {
    public double getPrice(int daysRented);
    public int getPoints(int daysRented);
}

```

- El estado concreto serían cada una de las categorías que implementan la interfaz Category.

SUBCLASE ESTADO CONCRETO – PELÍCULA DE NUEVO LANZAMIENTO:

```

public class NewReleaseMovie implements Category
{
    @Override
    public double getPrice(int daysRented) {
        return daysRented * 3;
    }
    @Override
    public int getPoints(int daysRented) {
        return (daysRented > 1) ? 2 : 1;
    }
}

```

SUBCLASE ESTADO CONCRETO – PELÍCULA REGULAR:

```

public class RegularMovie implements Category
{
    @Override
    public double getPrice(int daysRented) { // ... }
    @Override
    public int getPoints(int daysRented) { // ... }
}

```


Práctica 3: Editor gráfico:

PATRONES TEMPLATE METHOD Y STATE

El **patrón Template** define el esqueleto de un algoritmo en una operación, definiendo algunos pasos hasta las subclases. Permite que estas redefinan ciertos pasos del algoritmo sin cambiar su estructura en sí.

La parte común se implementa en la clase padre y la parte que varía se implementa en un método común que será **protected** e implementarán los hijos. Estamos haciendo esto en la **clase abstracta Herramienta Creación**.

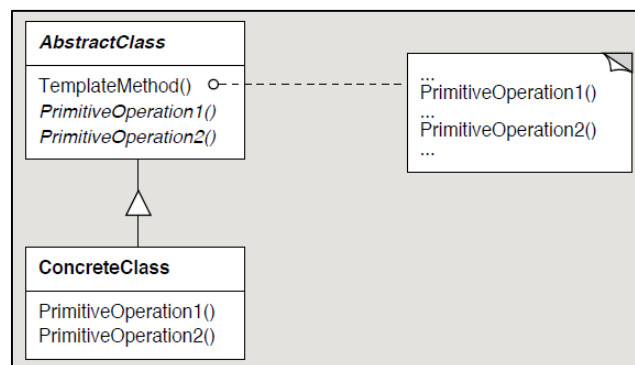
En el método **soltar** definimos el esqueleto del algoritmo que llama al método **doCrearFigura**. Definimos el método **abstracto doCrearFigura** que será el que habrá que implementar en las subclases, que son las que saben el tipo de figura que se deberá crear.

```
public abstract class HerramientaCreacion implements Herramienta
{
    public void soltar(int x, int y) {
        fin = new Point(x, y);
        Figura figura = doCreaFigura(inicio, fin);
        editor.getDibujo().AddFigura(figura);
        editor.finHerramienta();
    }

    protected abstract Figura doCreaFigura(Point inicio, Point fin);
}

public class HerramientaCirculo extends HerramientaCreacion
{
    protected Figura doCreaFigura(Point inicio, Point fin) {
        Point centro = new Point((inicio.x + fin.x) / 2, (inicio.y +
            fin.y) / 2);
        int radio = Math.max(fin.x - inicio.x, fin.y - inicio.y) / 2;
        return new Circulo(centro, radio);
    }
}
```

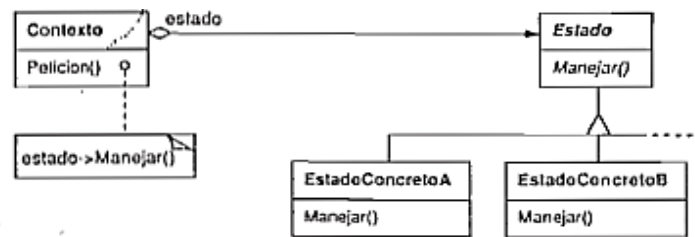
Contexto:



- **Clase Abstracta** sería la clase **Herramienta Creación**.
- **Clase Concreta** serían las clases **Herramienta Cuadrado y Herramienta Círculo**.

En esta práctica el **State** es la Herramienta: añadir distintas herramientas será añadir distintas implementaciones de los métodos mover, pinchar, soltar.

Contexto:



- El **editor es el contexto**. Además, mantiene una referencia a una subclase de estado concreto que representa el estado actual: atributo actual de tipo herramienta. Delega las operaciones dependientes de la herramienta (pinchar, mover y soltar) al objeto que representa el estado actual.

```

public class Editor
{
    private Dibujo dibujo;
    private Herramienta actual;
    private Herramienta principal;

    public void setHerramienta(Herramienta herramienta) {
        if (herramienta != null)
            this.actual = herramienta;
    }
    public void pinchar(int x, int y) {
        actual.pinchar(x, y);
    }
    public void mover(int x, int y) {
        actual.mover(x, y);
    }
    public void soltar(int x, int y) {
        actual.soltar(x, y);
    }
}
  
```

- El **estado es la interfaz Herramienta**: define la interfaz para encapsular el comportamiento asociado con el estado del contexto.

```

public interface Herramienta {
    public void pinchar(int x, int y);
    public void mover(int x, int y);
    public void soltar(int x, int y);
}
  
```

- Las subclases **estado concreto son las herramientas** (creación y selección).

```

public abstract class HerramientaCreacion implements Herramienta
{
    public void pinchar(int x, int y) {
        inicio = new Point(x, y);
    }
    public void mover(int x, int y) {
    }
    public void soltar(int x, int y) {
        fin = new Point(x, y);
        Figura figura = doCreaFigura(inicio, fin);
        editor.getDibujo().AddFigura(figura);
        editor.finHerramienta();
    }
}
  
```

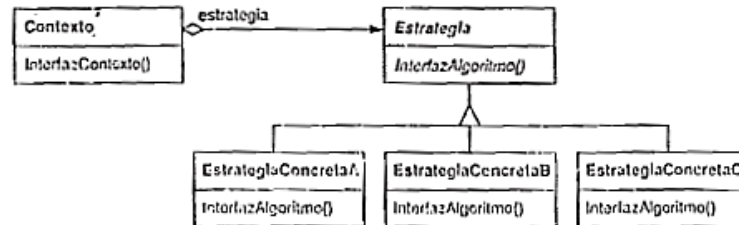
```
public class HerramientaSeleccion implements Herramienta
{
    private Point posicion
    public void pinchar(int x, int y) {
        seleccionada = editor.getDibujo().getFigura(x, y);
        posicion = new Point(x, y);
    }
    public void mover(int x, int y) {
        mueveIncremento(x, y);
    }
    public void soltar(int x, int y) {
        mueveIncremento(x, y);
    }
}
```

Práctica 4: Formularios:

PATRONES STRATEGY Y COMPOSITE

El **patrón Strategy** define una familia de algoritmos, encapsula cada uno de ellos y los hace intercambiables.

Contexto:



- **Contexto:** sería la clase **campo**. Mantiene una referencia a un objeto estrategia que guardamos en el atributo **validación**.

```
public class Campo
{
    private String texto;
    private String etiqueta;
    private Validation validacion;

    public void pideDato() {
        BufferedReader consola = new BufferedReader(new
            InputStreamReader(System.in));
        do {
            try {
                System.out.print(etiqueta + ": ");
                texto = consola.readLine();
            } catch (IOException ex) {
                System.out.println(ex);
            }
        } while (!validacion.isValid(texto));
    }
}
```

- **Estrategia:** sería la clase **validation**. Declara una interfaz común a todos los algoritmos permitidos. El contexto usa esta interfaz para llamar al algoritmo definido por una clase que implementan validation.

```
public interface Validation {
    public boolean isValid(String texto);
}
```

- **Estrategia Concreta:** son todas aquellas clases que implementan algoritmos utilizando la interfaz estrategia. En este caso **todas las clases que implementan validation**.

```
public class CheckNumber implements Validation {

    public boolean isValid(String texto) {
        for (char ch : texto.toCharArray())
            if (!Character.isDigit(ch))
                return false;
        return true;
    }
}
```

```

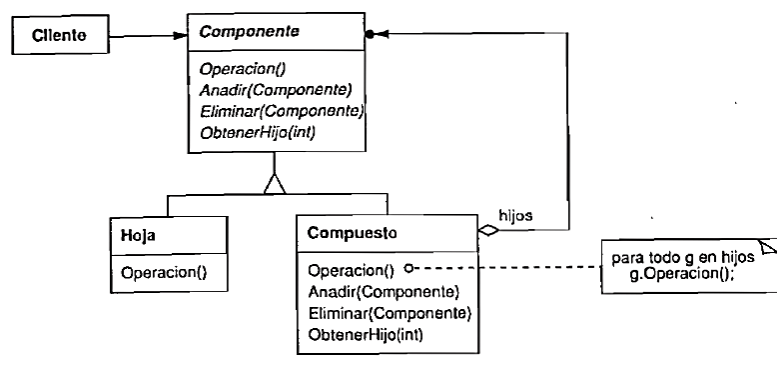
public class CheckText implements Validation {

    public boolean isValid(String texto) {
        // ...
    }
}

```

El **patrón Composite** permite componer objetos en estructuras arbóreas para representar jerarquías todo-parte.

Contexto:



- El **cliente sería campo**. Manipula los objetos de la composición a través de la interfaz componente, en este caso, validation.
- El **componente sería validation**. Declara la interfaz común.
- El **compuesto serían las clases checkAll y checkAny**. Almacenan los componentes hijos en atributos de tipo List<Validation>.

```

public class CheckAll implements Validation

    private List<Validation> validations;

    public CheckAll(Validation... validations) {
        this.validations = Arrays.asList(validations);
    }

    public boolean isValid(String texto) {
        for (Validation validacion : validations)
            if (!validacion.isValid(texto))
                return false;
        return true;
    }
}

```

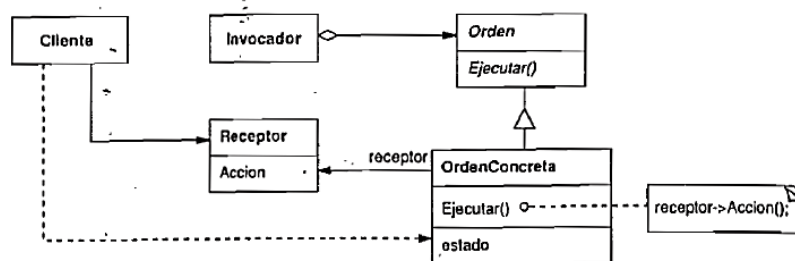
- **Hoja: todas las validaciones primitivas.**

Práctica 5: Rehacer/Deshacer:

PATRÓN COMMAND

El **patrón Command** encapsula una petición dentro de un objeto.

Contexto:



- **Orden:** es la interfaz **cambio**. Define una interfaz para ejecutar una operación, en este caso Undo y Redo.

```
public abstract interface Change {  
  
    public abstract void undo();  
    public abstract void redo();  
}
```

- **Orden Concreta:** son las clases **cambioCreacion y cambioMovimiento**. Asocia un objeto receptor con una acción.

```
/* */ public class CreateFigureChange implements Change {  
/* */     private editor.Figura figura;  
/* */     private Dibujo dibujo;  
/* */  
/* */     public CreateFigureChange(Dibujo dibujo, editor.Figura figura) {  
/* 11 */         this.figura = figura;  
/* 12 */         this.dibujo = dibujo;  
/* */     }  
/* */  
/* */     public void undo() {  
/* 16 */         this.dibujo.removeFigura(this.figura);  
/* */     }  
/* */  
/* */     public void redo() {  
/* 20 */         this.dibujo.addFigura(this.figura);  
/* */     }  
/* */ }  
  
/* */ public class MovementChange implements Change {  
/* */     private Figura figura;  
/* */     private int dx;  
/* */     private int dy;  
/* */  
/* */     public MovementChange(Figura figura, int dx, int dy) {  
/* 12 */         this.figura = figura;  
/* 13 */         this.dx = dx;  
/* 14 */         this.dy = dy;  
/* */     }  
/* */ }
```

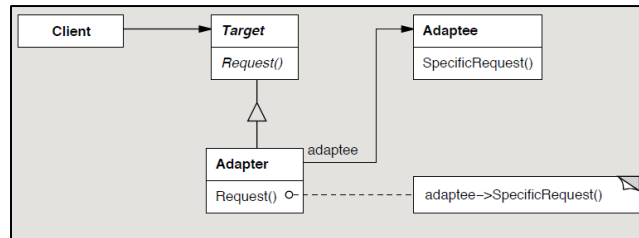
```
/*      */ public void undo()
/*      */ {
/* 19 */   this.figura.mover(-this.dx, -this.dy);
/*      */ }
/*      */
/*      */ public void redo()
/*      */ {
/* 24 */   this.figura.mover(this.dx, this.dy);
/*      */ }
/*      */ }
```

Práctica 6: Mapa:

PATRÓN ADAPTER

El **patrón Adapter** convierte una clase en otra interfaz que es la que esperan los clientes. Permite que cooperen clases que de otra forma no podrían por tener interfaces incompatibles.

Contexto:



- **Target:** sería la interfaz **Marcador**. Define la interfaz específica del dominio.

```
public interface Marcador {
    Coordenadas getCoordenadas();

    String getTooltipInfo();

    void abrir();
}
```

- **Adapter:** serían las clases **AdaptadorMarcadorFoto**, **AdaptadorMarcadorMonumento** y **AdaptadorMarcadorRestaurante**. Adaptan la interfaz de Adaptee a la de Target.

```
public class AdaptadorMarcadorFoto implements Marcador {
    private Foto foto;

    @Override
    public Coordenadas getCoordenadas() { foto.getCoordenadas() }

    @Override
    public String getTooltipInfo() { foto.getUsuario() }

    @Override
    public void abrir() { foto.descargar() }
}
```

- **Adaptee:** serían las clases **Foto**, **Monumento** y **Restaurante**. Son las clases existentes cuya interfaz necesita ser adaptada.

```
public class Foto {
    private String usuario; // Usuario que ha subido la foto
    private String descripción;
    private Coordenadas coordenadas; // Coordenadas GPS de la foto

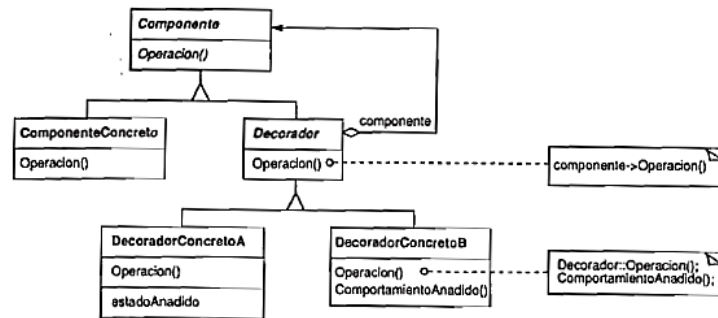
    public String getUsuario() { }
    public Coordenadas getCoordenadas() { }
    public void descargar() { }
}
```


Práctica 7: Sistema de ficheros:

PATRÓN DECORATOR

El **patrón Decorator** permite añadir responsabilidades al objeto de forma dinámica, proporcionando una alternativa flexible a la herencia para extender funcionalidad. Los decoradores cumplen la interfaz del objeto original y son transparentes para el cliente.

Contexto:



- **Componente:** sería la interfaz **output**. Define la interfaz de los objetos a los que se les puede añadir responsabilidades dinámicamente.

```
public interface Output {
    void send(char c) throws IOException;
    void close() throws IOException;
}
```

- **Componente concreto:** serían las clases **BluetoothOutput**, **FileOutput** e **InternetOutput**.

```
public class BluetoothOutput implements Output {
    private StringWriter stringWriter;

    public BluetoothOutput(String device) {
        stringWriter = new StringWriter();
        stringWriter.append("\n--- START Bluetooth[" + device +
            "]\n");
    }

    public void send(char c) throws IOException {
        stringWriter.append(c);
    }

    public void close() throws IOException {
        System.out.print(stringWriter.toString());
        System.out.println("--- END Bluetooth");
    }
}
```

- **Decorador Concreto:** serían las clases que añaden responsabilidad: **Contador**, **EliminadorEspacios**, **Contador...**

```
public class Contador implements Output {
    private Output output;
    private int counter;
```

```
    public void send(char c) throws IOException {  
        output.send(c);  
        counter++;  
    }  
  
    public int getCounter() {  
        return counter;  
    }  
  
    public void close() throws IOException {  
        output.close();  
    }  
}
```

Práctica 8: Sistema de encuestas:

PATRÓN OBSERVER

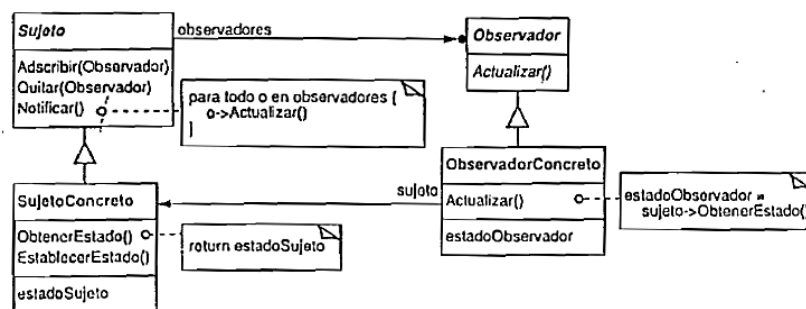
El **patrón Observer** define una dependencia uno a muchos entre objetos de modo que cuando un objeto cambia su estado, todos los demás objetos dependientes se modifican y actualizan automáticamente.

En el entorno de la encuesta la única responsabilidad es llevar cuenta de los votos. Los gráficos tienen que ser actualizados cuando la encuesta llegue a un nuevo voto.

El patrón Observer hace que la encuesta sea un objeto capaz de comunicarse con los gráficos.

Cuando cambia el número de votos, los gráficos, que están pendientes del cambio, se actualizan automáticamente.

Contexto:



- **Sujeto:** sería la clase **pregunta**. Conoce a sus observadores (`List<VoteObserver>`). Utiliza el método `avisaAObservers()` cuando se incrementan los votos recibidos.

```
public class Pregunta
{
    private int si, no;
    private String pregunta;
    private List<VoteObserver> observers = new ArrayList<>();

    private void avisaAObservers() {
        for (VoteObserver voteObserver : observers)
            voteObserver.votoRecibido(this);
    }

    public void addObserver(VoteObserver observer) {
        observers.add(observer);
    }

    public void removeObserver(VoteObserver observer) {
        observers.remove(observer);
    }

    public void incrementaSi() {
        si++;
        avisaAObservers();
    }

    public void incrementaNo() {
        no++;
        avisaAObservers();
    }
}
```

- **Observador:** sería la clase **voteObserver**. Define una interfaz para actualizar objetos que deben ser notificados ante cambios en el sujeto.

```
public interface VoteObserver {
    void votoRecibido(Pregunta encuesta);
}
```

- **Observador concreto:** serían las clases **GraficoCircular, LineaEstado, GraficoBarras...** Guardan un estado que debería ser consistente con el del sujeto. Implementan la interfaz de actualización del Observer.

```
public class GraficoCircular implements VoteObserver

    @Override
    public void votoRecibido(Pregunta encuesta) {
        System.out.println("Dibujando gráfico circular");
    }

public class LineaDeEstado implements VoteObserver {

    @Override
    public void votoRecibido(Pregunta encuesta) {
        System.out.println("Si = " + encuesta.getVotosSi() + "/ No
        = " + encuesta.getVotosNo());
    }
}
```

Práctica 10: Ball game:

Originalmente tenemos:

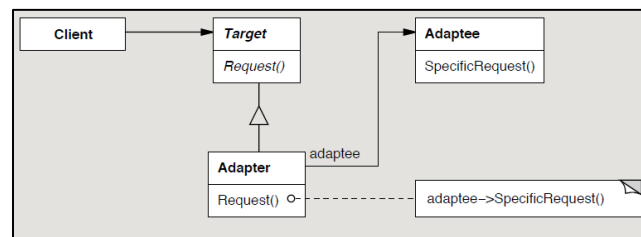
```
enum Platform {  
    ANDROID, WINDOWS, PLAYSTATION  
};  
  
public class BallGame {  
    // Seleccionar para qué plataforma se quiere generar el juego  
    private Platform platform = Platform.ANDROID;  
    // private Platform platform = Platform.WINDOWS;  
    // private Platform platform = Platform.PLAYSTATION;  
  
    private AndroidAPI android;  
    private WindowsAPI windows;  
    private Playstation5API playstation;  
  
    private void setAPI() {  
        if (platform == Platform.ANDROID) <--  
            android = new AndroidAPI();  
        else if (platform == Platform.WINDOWS) <--  
            windows = new WindowsAPI();  
        else <--  
            playstation = new Playstation5API();  
    }  
}
```

Se mantiene la misma lógica condicional en todos los métodos del juego en función de la api que estemos utilizando.

V1. PATRÓN ADAPTER

El **patrón Adapter** convierte una clase en otra interfaz que es la que esperan los clientes. Permite que cooperen clases que de otra forma no podrían por tener interfaces incompatibles.

Contexto:



- **Target:** sería la interfaz **platformAdapter**. Define la interfaz específica del dominio.

```
public interface PlatformAdapter {  
    Image2D loadImage(String file);  
    void drawBall(Image2D image, Point point);  
    Point getPosition();  
}
```

- **Adapter:** serían la clases **AndroidAdapter, WindowsAdapter, PlayStationAdapter**. Adaptan la interfaz de Adaptee a la de Target.

```
public class WindowsAdapter implements PlatformAdapter {
    private WindowsAPI windows;

    @Override
    public Image2D loadImage(String file) {
        return windows.loadFile(file);
    }

    @Override
    public void drawBall(Image2D image, Point point) {
        windows.paint(point.x, point.y, image);
    }

    @Override
    public Point getPosition() {
        return windows.getMouseClick();
    }
}
```

- **Adaptee:** serían las clases **AndroidApi, WindowsApi, PlayStationApi**. Son las clases existentes cuya interfaz necesita ser adaptada.

```
public class WindowsAPI {
    private Point point = new Point(0, 0);

    public Image2D loadFile(String name) {
        System.out.println("(Windows) Cargada imagen '" + name +
            "' del disco duro.");
        return new Image2D(name, 10, 10);
    }

    public void paint(int x, int y, Image2D image) {
        System.out.println("(Windows) Dibujando '" +
            image.getName() + "' en [" + x + ", " + y + "]");
    }

    public Point getMouseClick() {
        point.translate(10, 10);
        return new Point(point);
    }
}
```

- La clase **BallGame** ahora sería:

```
public class BallGame {

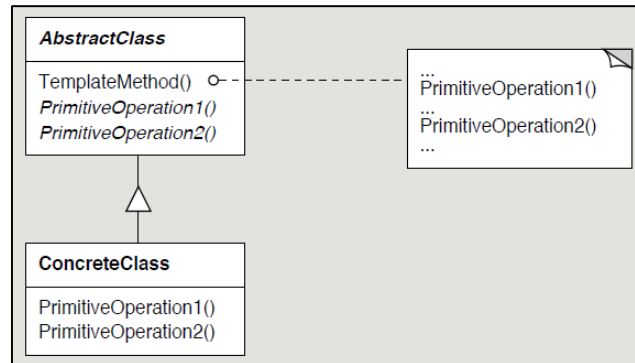
    public void play(PlatformAdapter adapter) {
        Image2D image = adapter.loadImage("Bola.jpg");

        for (int i = 0; i < 10; i++) {
            Point point = adapter.getPosition();
            adapter.drawBall(image, point);
        }
    }
}
```

V2. PATRÓN TEMPLATE

El **patrón Template** define el esqueleto de un algoritmo en una operación, definiendo algunos pasos hasta las subclases. Permite que estas redefinan ciertos pasos del algoritmo sin cambiar su estructura en sí.

Contexto:



- **AbstractClass:** sería la clase **BallGame**.

```
public abstract class BallGame {

    public void play() {
        Image2D image = loadImage("Bola.jpg");
        for (int i = 0; i < 10; i++) {
            Point point = getPosition();
            drawBall(image, point);
        }
    }

    public abstract Image2D loadImage(String file);
    public abstract void drawBall(Image2D image, Point point);
    public abstract Point getPosition();
}
```

- **ConcreteClass:** serían las clases **AndroidGame**, **PlayStationGame** y **WindowsGame**.

```
public class AndroidGame extends BallGame {
    private AndroidAPI android;

    @Override
    public Image2D loadImage(String file) {
        return android.loadResource(file);
    }

    @Override
    public void drawBall(Image2D image, Point point) {
        android.draw(point.x, point.y, image);
    }

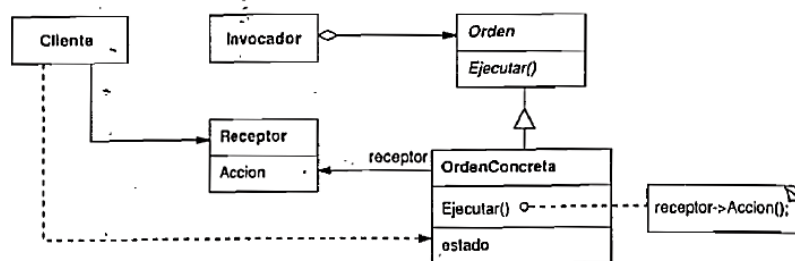
    @Override
    public Point getPosition() {
        return android.getTouch();
    }
}
```

Práctica Simulacro examen:

PATRONES COMMAND Y COMPOSITE

El **patrón Command** encapsula una petición dentro de un objeto.

Contexto:



- **Orden:** sería la interfaz **Acción**.

```
public interface Accion {
    public void execute(Editor editor);
}
```

- **OrdenConcreta:** serían la clases **Abrir, BorrarUltimaPalabra, Insertar, Macro, Reemplazar**.

```
public class Reemplazar implements Accion {
    private String l1;
    private String l2;

    public Reemplazar(String l1, String l2) {
        this.l1 = l1;
        this.l2 = l2;
    }

    @Override
    public void execute(Editor editor) {
        editor.setTexto(new
        StringBuilder(editor.getTexto().toString().replaceAll(Patt
        ern.quote(l1), l2)));
    }
}
```

- **Receptor:** sería la clase **Editor**.

```
public class Editor
    private StringBuilder texto;
    private ManejadorAcciones acciones;

    public void abrir(String fichero) {
        this.acciones.execute(new Abrir(fichero));
    }

    new Abrir(fichero) --> orden concreta

    public void borrarUltimaPalabra() {
        this.acciones.execute(new
        BorrarUltimaPalabra(getIndexUltimaPalabra()));
    }
}
```



```

public void insertar(String[] cadena) {
    this.acciones.execute(new Insertar(cadena));
}

public void reemplazar(String l1, String l2) {
    this.acciones.execute(new Reemplazar(l1, l2));
}

public void grabar(String nombreMacro) {
    this.acciones.grabar(nombreMacro);
}

public void pararGrabacion() {
    this.acciones.parar();
}

public void ejecutarMacro(String nombreMacro) {
    this.acciones.executeMacro(nombreMacro);
}

```

- **Invocador:** sería la clase **ManejadorAcciones**.

```

public class ManejadorAcciones {
    private Editor editor;
    private boolean grabando;
    private Macro macro;
    private Map<String, Macro> macros;

    public void execute(Accion accion) {
        accion.execute(this.editor);
        if (grabando)
            macro.addAccion(accion);
    }

    public void executeMacro(String nombreMacro) {
        this.execute(macros.get(nombreMacro));
    }

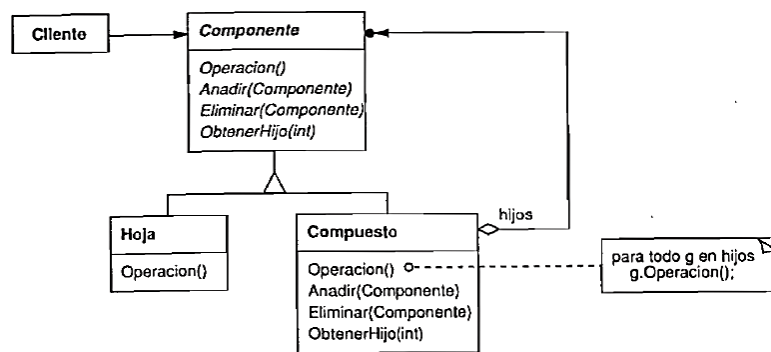
    public void grabar(String nombreMacro) {
        this.grabando = true;
        this.macro = new Macro(nombreMacro);
    }

    public void parar() {
        if (!grabando)
            return;
        this.grabando = false;
        macros.put(this.macro.getNombre(), this.macro);
    }
}

```

El **patrón Composite** permite componer objetos en estructuras arbóreas para representar jerarquías todo-parte.

Contexto:



- **Componente:** sería la interfaz **Acción**.

```
public interface Acción {  
    public void execute(Editor editor);  
}
```

- **Hoja:** serían las **Órdenes Concretas**.
- **Compuesto:** sería la clase **Macro**.

```
public class Macro implements Acción {  
    private String nombre;  
    private List<Acción> acciones;  
  
    public Macro(String nombre) {  
        this.nombre = nombre;  
        this.acciones = new ArrayList<Acción>();  
    }  
  
    public String getNombre() {  
        return this.nombre;  
    }  
  
    public void addAction(Acción a) {  
        this.acciones.add(a);  
    }  
  
    @Override  
    public void execute(Editor editor) {  
        for(Acción a : acciones)  
            a.execute(editor);  
    }  
}
```